

Trabalho — Microclima Local e Simulação de Evacuação

PARTE 1 — O Algoritmo do Microclima Local

Dados Coletados

Três pontos foram escolhidos no trajeto casa–faculdade, medidos em dois horários (manhã ~8h e tarde ~15h). Os dados de Temperatura (°C), Umidade (%) e IQA foram obtidos em sites de meteorologia local e medidores independentes.

Faixas Oficiais de IQA — CETESB/Brasil

Faixa	Classificação
0 – 40	Boa (sem risco à saúde)
41 – 80	Moderada (sensíveis devem reduzir esforços ao ar livre)
81 – 120	Ruim (toda a população pode sentir efeitos)
121 – 200	Muito Ruim (evitar atividades ao ar livre)
201 – 299	Péssima (emergência de saúde pública)

Código Python — Análise de Microclima

```
# Dados: [Local, Temp(°C), Umidade(%), IQA]

dados_manha = [
    ["Praca Arborizada", 24, 72, 35],
    ["Cruzamento Movimentado", 27, 58, 95],
    ["Ponto de Onibus", 26, 61, 68],
]

dados_tarde = [
    ["Praca Arborizada", 28, 65, 42],
    ["Cruzamento Movimentado", 33, 44, 130],
    ["Ponto de Onibus", 31, 48, 88],
]

def classificar_iqa(iqa):
    match True:
        case _ if iqa <= 40:
            return "Boa"
        case _ if iqa <= 80:
            return "Moderada"
        case _ if iqa <= 120:
            return "Ruim"
        case _ if iqa <= 200:
            return "Muito Ruim"
        case _:
            return "Pessima"

def calcular_conforto(temp, umidade, iqa):
    """Nota de Conforto Urbano (0-10): penaliza desvios de temperatura,
    umidade fora de 50-70% e IQA elevado."""

    temp_ideal = 22.5
    penalidade_temp = min(abs(temp - temp_ideal) * 0.4, 4.0)

    if 50 <= umidade <= 70:
        penalidade_umid = 0.0
    else:
        desvio = min(abs(umidade - 50), abs(umidade - 70))
        penalidade_umid = min(desvio * 0.1, 3.0)

    penalidade_iqa = min((iqa / 300) * 3, 3.0)

    nota = 10.0 - penalidade_temp - penalidade_umid - penalidade_iqa

    return round(max(nota, 0.0), 2)

def analisar_microclima(periodo, dados):
```

```

print(f"\n{'='*55}")
print(f" ANALISE DE MICROCLIMA - {periodo.upper()}")
print(f"\n{'='*55}")

metricas = ["Temperatura (C)", "Umidade (%)", "IQA"]

for local_dados in dados:

    nome = local_dados[0]
    valores = local_dados[1:]

    print(f"\nLocal: {nome}")

    for i, metrica in enumerate(metricas):
        print(f" {metrica}: {valores[i]}")

    temp, umidade, iqa = valores

    qualidade = classificar_iqa(iqa)
    nota = calcular_conforto(temp, umidade, iqa)

    print(f" Classificacao IQA: {qualidade}")
    print(f" Nota de Conforto : {nota}/10")

    if nota >= 7:
        print(" OK - Confortavel para atividades ao ar livre.")
    elif nota >= 4:
        print(" ATENCAO - Conforto moderado.")
    else:
        print(" ALERTA - Evite exposicao prolongada.")

def main():

    analisar_microclima("Manha (08h)", dados_manha)
    analisar_microclima("Tarde (15h)", dados_tarde)

    todos = dados_manha + dados_tarde

    notas = [(d[0], calcular_conforto(d[1], d[2], d[3])) for d in todos]

    melhor = max(notas, key=lambda x: x[1])
    pior = min(notas, key=lambda x: x[1])

    print(f"\nMelhor ponto: {melhor[0]} - Nota {melhor[1]}")
    print(f"Pior ponto : {pior[0]} - Nota {pior[1]}")

main()

```

PARTE 2 — Navegação e Evacuação Espacial

Mapa Físico Levantado

- Nó 0 — Quarto (ponto de partida): caminho_livre + chave disponível
- Nó 1 — Corredor: porta_trancada (requer chave)
- Nó 2 — Sala: extintor visível, caminho livre
- Nó 3 — Cozinha: caminho_livre
- Nó 4 — Saída (porta da rua): destino final

Código Python — Simulador de Evacuação

```
nos = [
    "Quarto (ponto de partida)",
    "Corredor",
    "Sala",
    "Cozinha",
    "Saida - Porta da Rua",
]

estados = [
    "caminho_livre",
    "porta_trancada",
    "extintor",
    "caminho_livre",
    "saida",
]

inventario = []
chave_disponivel = 0
chave_nome = "chave_corredor"

ENERGIA_MAXIMA = 10

def tentar_avancar(posicao, energia):

    proximo = posicao + 1
    estado_proximo = estados[proximo]

    if estado_proximo == "porta_trancada":

        print(" Porta trancada a frente.")

        if chave_nome in inventario:
            print(" Agente usou a chave! Avancando...")
            inventario.remove(chave_nome)
            return proximo, energia - 1

        else:
            print(" Sem chave. Agente recua para procurar.")
            return posicao - 1, energia - 2

    elif estado_proximo == "extintor":
        print(" Extintor visível. Caminho livre. Avancando.")
        return proximo, energia - 1

    elif estado_proximo == "caminho_livre":
        print(" Caminho livre. Avancando.")
        return proximo, energia - 1

    elif estado_proximo == "saida":
        print(" SAIDA ALCANÇADA!")
        return proximo, energia

    return posicao, energia - 1

def simular_evacuacao():

    print("=" * 50)
    print(" SIMULADOR DE EVACUACAO - AGENTE AUTONOMO")
    print("=" * 50)

    posicao = 0
    energia = ENERGIA_MAXIMA
    evacuado = False

    print(f"\nAgente pega a '{chave_nome}' no quarto.")
    inventario.append(chave_nome)

    while energia > 0 and not evacuado:
        print(f"\nLocal: [{nos[posicao]}]")
```

```
print(f"Estado: {estados[posicao]}")
print(f"Energia: {energia}")
print(f"Inventario: {inventario if inventario else 'vazio'}")

if posicao < 0:
    print("\nAgente desorientado. Falha.")
    break

if estados[posicao] == "saida":
    print("\nEVACUACAO BEM-SUCEDIDA!")
    evacuado = True
    break

nova_posicao, nova_energia = tentar_avancar(posicao, energia)

if nova_posicao < 0:
    print("\nSem para onde recuar. Falha na evacuacao.")
    break

posicao = nova_posicao
energia = nova_energia

if not evacuado and energia <= 0:
    print("\nEnergia esgotada! Evacuacao falhou.")

print(f"\nResultado: {'EVACUADO' if evacuado else 'FALHOU'}")
print(f"Energia restante: {energia}")

simular_evacuacao()
```

PARTE 3 — Reflexão Crítica

Parágrafo 1:

Mapear fisicamente a casa revelou que o maior desafio não foi percorrer o espaço, mas traduzi-lo. Cada detalhe — a porta que trava, a chave num determinado cômodo — precisou virar uma variável discreta. Esse processo de abstração forçada mostrou que programar é, antes de tudo, decidir o que importa e o que pode ser simplificado sem comprometer a lógica do problema.

Parágrafo 2:

Ao transformar corredores em listas e obstáculos em condições if/else, percebi que cada decisão carrega consequências encadeadas — a chave só vale se o agente passou pelo quarto antes. Esse raciocínio de estado e dependência é exatamente o que o loop while e o if aninhado treinam: enxergar não só o problema atual, mas tudo que levou até ele.

Parágrafo 3:

Fora da tela, esse hábito se manifesta naturalmente. Diante de qualquer situação complexa — um conflito, uma decisão importante — busco identificar as variáveis, as condições necessárias para avançar e o que já tenho no 'inventário'. Programar ensina que nenhum problema é monolítico: ele sempre pode ser quebrado em partes menores, testadas e reconectadas com mais clareza.